

Shrinking the Feasible Family: Answer-Preserving Reduction Techniques for Set-Theoretic Estimation

A methods note, grounded in the Lean 4 mechanization of project `thejeb`

Project `thejeb`

July 28, 2026

Abstract

In set-theoretic estimation (STE) a problem is a family of property sets S_i over a solution space, and its answer object is the feasibility set $\mathcal{N} = \bigcap_i S_i$ — the family of hypotheses consistent with every piece of information [4]. Generating that family is step one of an STE attack. This note addresses the step that follows: once the (possibly enormous) feasible family exists, how does one shrink, restrict, collapse, certify, and structurally compress it while provably preserving the STE answers — feasibility, and the must / may / cannot queries quantified over feasible hypotheses? We synthesize the project’s machine-checked toolkit into a reusable technique: exact property-set application (the STE meet), contravariant restriction across corpus states, garbage collection of the variable universe, the must-quotient as the only generally safe deterministic collapse, minimal supports as shrunken certificates, gluing of local families with an exact cross-constraint obstruction, and — past the proven 2^N extensional-state wall, where generic reduction provably stops — width-bounded structural compression by variable elimination, junction trees, and factorization, down to Carlson’s substitution reduction of cipher classes [2]. Every guarantee is cited to a named Lean 4 theorem in the project source; qualifications (consistency premises, the compatibility notion for gluing, the exact scope of the lower bounds) are stated where the mechanization states them.

1 Setup: the feasible family and the queries to preserve

An STE problem over a solution space Ω is an indexed family of property sets $S : I \rightarrow \mathcal{P}(\Omega)$, one per piece of information; the feasibility set is

$$\mathcal{N} = \bigcap_{i \in I} S_i,$$

and a set-theoretic estimate is any point of $\mathcal{N}$ [4, §II-C]. In the mechanization this is `feasibilitySet` with the membership law `mem_feasibilitySet`: a hypothesis is feasible iff it lies in every property set. Enforcing only a subfamily $J \subseteq I$ yields the partial feasibility set `partialFeasibilitySet`, and information monotonicity — more information never enlarges the feasible family — is the antitonicity theorem `partialFeasibilitySet_antitone`. If the full intersection is empty, at least one piece of information is unfair for every candidate truth (`exists_unfair_of_feasibilitySet_eq_empty`): inconsistency is not a failure mode but a detection principle.

The feasible family is rarely wanted for its own sake. What an STE consumer asks of it are *queries*:

- **Feasibility:** is $\mathcal{N} \neq \emptyset$ (the formulation is consistent)?
- **Must:** does a relation hold in *every* feasible hypothesis?
- **May:** does it hold in *some* feasible hypothesis?
- **Cannot:** does it fail in every feasible hypothesis?

The project’s working instantiation is the dynamic-frame model `STE.DynamicFrame.Model`: documents carry claims, constraints carry provenance supports, a corpus D activates exactly the constraints whose support lies in D , and the feasible family `Model.feasible` is the partial feasibility set of the active property sets. Each feasible hypothesis carries an equivalence relation on claims, and the queries become `MustSame`, `MaySame`, `CannotSame`, and the explicit two-sided marker `Uncertain` (both merge and split remain feasible). Everything below is stated for this model but the technique is generic: any STE problem whose hypotheses support a relational query has the same shape.

The goal of this note is *answer-preserving reduction*: replace the feasible family, or its representation, by something smaller, while provably returning the same answers to the queries above. Reductions come in three kinds — reductions that shrink the family itself (Sections 2–3), reductions that compress a query-equivalent summary of it (Sections 4–6), and reductions that compress its representation (Sections 7–8).

2 Base reduction: the exact meet

The primitive shrinking step is property-set application: intersect the current family X with the property set of one constraint k ,

$$X \mapsto X \cap S_k.$$

This is `applyConstraint` in `Ste.DynamicFrame.Solver`, and the mechanization proves that this single operation already has every algebraic property a practitioner needs to schedule it freely:

- **Monotone shrinking.** Application only removes hypotheses (`applyConstraint_subset`); at the corpus level, activating more documents only shrinks the feasible family (`feasible_antitone`).
- **Idempotence.** Applying the same property set twice equals applying it once (`applyConstraint_idempotent`): re-running an audited constraint is free.
- **Order-independence.** Applications commute (`applyConstraint_comm`); a finite run depends only on the *set* of constraints applied, not their order or multiplicity (`solve_congr`, `solve_perm`).
- **Exactness.** A finite run is extensionally the partial feasibility set of the applied constraints (`solve_eq_partialFeasibilitySet`), and if the schedule enumerates exactly the active constraints of corpus D , the solver output *is* the model’s feasible family (`solve_eq_feasible`). There is no approximation layer between “run the solver” and “the STE answer”.
- **Delta decomposition.** Each application splits the state exactly into survivors and casualties: `apply_union_reduction` proves $X = (X \cap S_k) \cup (X \setminus (X \cap S_k))$, and `mem_solverReduction` identifies the discarded set as precisely the current candidates violating the new constraint. A constraint causes no reduction iff the current family already refines it (`solverReduction_eq_empty_iff`) — the operational test for a redundant constraint.

When to reach for it. Always. This is the sound kernel of every other technique: any pipeline stage that cannot be expressed as (or proven equal to) a meet with an audited property set is where soundness must be argued separately. The delta decomposition additionally gives exact incremental accounting — what each constraint killed — for free.

3 Restriction across instances: garbage-collecting the universe

An STE attack rarely runs on one fixed instance. Corpora grow and shrink; the practitioner wants to reuse a computed feasible family rather than regenerate it. The mechanization gives this a precise shape: for corpora $D \subseteq E$, the map that views an E -feasible hypothesis as a D -feasible one (`restrictFeasible`) satisfies the identity and composition laws `restrictFeasible_id` and `restrictFeasible_comp` — so the assignment $D \mapsto \text{feasible}(D)$ is a contravariant, presheaf-shaped construction on the poset of corpora, with the same laws holding for the packaged world views (`restrictWorld_id`, `restrictWorld_comp`). The image of restriction is characterized exactly: a small-corpus hypothesis is in the image iff the same ambient hypothesis is feasible on the larger corpus (`mem_range_restrictFeasible_iff`).

Two consequences matter for shrinking:

Incremental deltas instead of regeneration. Growing the corpus decomposes exactly: a hypothesis is feasible on $E \supseteq D$ iff it is feasible on D and satisfies each *newly activated* constraint (`mem_feasible_extension`, with the new constraints isolated as `newConstraints`). If an insertion activates nothing new, the family is unchanged (`feasible_eq_of_no_new_constraints`). So the already-shrunk family is the correct starting point for the meet of Section 2, never a lossy cache.

Partial restriction of the variable universe. The claim universe itself can be garbage-collected. Restricting active claims under corpus deletion is deliberately *partial*: a claim restricts iff its source document remains active, and when the restriction exists it is unique (`claimRestriction_exists_iff`, `claimRestriction_unique` — restriction fibers have size 0 or 1). Dropping the claims of deleted documents loses no explanatory power over the surviving universe, because local coreference is preserved definitionally under world restriction and claim inclusion (`localSame_restrict`).

Remark 1 (Retraction asymmetry). *Restriction runs against a one-way door. Adding documents induces a covariant map on the safe quotients (`canonicalMap`, built from `includeActiveClaim` and the monotonicity `mustSame_mono`); that map may identify old classes and is not generally injective. In the deletion direction there is in general no quotient map at all: a pair forced together on E can become uncertain on $D \subset E$, and mapping an E -class backward would require splitting it, which is not a function on classes. Deletion therefore cannot be implemented as the inverse of a merge history; it must be implemented semantically, by re-restricting the feasible family — which the laws above make exact.*

When to reach for it. Whenever instances are related by inclusion: streaming corpora, leave-one-out audits, provenance ablations. Restrict, do not recompute; garbage-collect claims of retired documents; and never try to undo a merge by bookkeeping.

4 Safe collapse: the must-quotient

The feasible family is a set of equivalence relations, and the practitioner wants *one* deterministic output. The only generally safe deterministic collapse is the intersection of the feasible relations: define $\text{Must}_D(c, d)$ to hold when every feasible hypothesis merges c and d . The mechanization proves

this intersection is itself an equivalence relation (`mustSame_equivalence`), packaged as the setoid `mustSetoid`, restricted to active claims as `activeMustSetoid`, with quotient type `CanonicalFrame`. Collapsing by `Must` therefore never invents an identification: it reports exactly the merges that survive in *all* feasible worlds.

The remaining pairs are not noise; they are classified. For a *consistent* corpus every pair is must-merged, cannot-merged, or explicitly uncertain (`coreference_exhaustive`); the deterministic classifier `corefStatus` returns `{must, cannot, uncertain, inconsistent}`, with the inconsistent case kept separate precisely because universal quantification over an empty feasible family is vacuous (`must_and_cannot_implies_inconsistent`). The statuses are stable in the useful direction: settled statuses persist under corpus growth (`mustSame_mono`, `cannotSame_mono`), while uncertainty can only be resolved, never created, by adding evidence (`uncertain_antitone`).

Why not collapse more aggressively — say, quotient by “some feasible world merges them”? Because the union of equivalence relations across feasible worlds is *not* an equivalence relation. The finite countermodel `Ste.DynamicFrame.Counterexample` exhibits three claims a, b, c and two feasible worlds (one merging $\{a, b\}$, one merging $\{b, c\}$) such that `May(a, b)` and `May(b, c)` hold while `Cannot(a, c)` holds: `maySame_not_transitive`. The uncertainty overlay fails transitivity the same way (`uncertain_not_transitive`). Any pipeline that transitively closes may-links or uncertain-links manufactures a merge ($a \sim c$) that every feasible hypothesis refutes.

Remark 2 (Qualification). *The must-quotient is safe on a consistent corpus. If the feasible family is empty, Must holds vacuously for every pair and the quotient degenerates; the mechanization guards this by carrying the consistency premise in `coreference_exhaustive` and by routing empty families to the separate `corefStatus` value `inconsistent`. Check feasibility before collapsing.*

When to reach for it. Whenever a downstream consumer demands a single partition. Emit the must-quotient plus the uncertain-pair list; never a transitive closure of possibilities.

5 Certification: minimal supports

A settled conclusion (Must or Cannot on some finite corpus) drags the whole corpus behind it as justification. That justification can be shrunk. Define `SupportsMust(D, c, d)`: the finite corpus D is consistent and forces c, d together (analogously `SupportsCannot`). The mechanization proves that every finitely supported conclusion has an *inclusion-minimal* finite subcorpus that still supports it: `exists_minimal_mustSupport` and `exists_minimal_cannotSupport`, instances of the general finite-minimization principle `exists_minimal_subfinset`. The minimal support is a certificate: a smallest-by-inclusion set of documents such that removing any one breaks the conclusion.

The mechanization is deliberately semantic here: the theorems assert existence of minimal supports, not that finding one is cheap. Note also that consistency is part of the support predicate — a subcorpus that forces the pair only vacuously (by being inconsistent) does not count as support.

When to reach for it. Explanation, audit, and caching. A minimal support is simultaneously the provenance answer (“why are these merged?”), the invalidation key for dynamic updates (the conclusion survives any deletion disjoint from its support), and a compressed proof obligation for human review.

6 Composition: gluing local families with an exact obstruction

Large problems are attacked in pieces: compute feasible families on subcorpora D and E , then combine. The obstruction to combining is exactly the genuinely cross-piece constraints. For an

ambient hypothesis h , `crossObstruction` collects the constraints whose support lies in $D \cup E$ but in neither piece alone and which h violates; the classification theorem `mem_feasible_union_iff` proves

$$h \in \text{feasible}(D \cup E) \iff h \in \text{feasible}(D) \wedge h \in \text{feasible}(E) \wedge \mathcal{O}_{D,E}(h) = \emptyset.$$

Two local world views are `Compatible` when they expose the same ambient hypothesis; a `GlobalExtension` is a world on $D \cup E$ restricting to both. Existence is settled exactly: `globalExtension_nonempty_iff` proves a global extension exists iff the views are compatible and the cross obstruction vanishes, with the witness constructed by `glue`. Uniqueness is a theorem, not a hope: the extension type is a subsingleton (`globalExtension_subsingleton`) — at most one global world extends a fixed pair of local worlds.

Remark 3 (Qualification). *Uniqueness is relative to the compatibility notion mechanized here: all views live in one ambient hypothesis type, and compatibility means literal equality of the underlying hypothesis. In a setting where local views are related by a looser correspondence, uniqueness would have to be re-proven for that correspondence. The obstruction set is computable in the finite case, but it is an obstruction per ambient hypothesis: gluing whole families means discharging it per surviving candidate.*

When to reach for it. Divide-and-conquer over provenance: shard the corpus, shrink each shard’s family locally (cheap), and pay the cross-constraint check only at the join. The theorem tells you exactly which constraints can obstruct — the straddlers — so the join cost is proportional to genuine coupling, not to corpus size.

7 The wall: 2^N extensional states, and why it is tight

How far can generic shrinking go? The finite mechanization `Ste.DynamicFrame.FiniteSolver` makes the ceiling exact. When the hypothesis universe is finite with $N = \text{maxWorlds}$ ambient worlds and satisfaction is decidable, the solver state is represented exactly by a finite set: filtering implements the meet (`coe_applyFinite`), and the executable run denotes precisely the abstract STE run (`coe_runFinite`, `coe_solveFinite`) — partial feasibility on the nose. Total reduction is bounded: a schedule can contain at most N properly shrinking steps (`reductionCost_le_maxWorlds`). The state space is 2^N exactly (`card_allExactStates`).

The bound is tight, and this is the pivot of the whole note: a single crisp property set can select *any* chosen subset of the universe (`arbitrary_subset_as_single_property`). Unrestricted crisp STE therefore realizes all 2^N extensional states, and no uniform sub-exponential family of states can be complete for it. No amount of clever generic machinery — meets, restriction, quotients, supports, gluing — improves the worst case, because the worst case is realizable by the inputs themselves. Generic reduction stops here. Everything further must buy its compression with *structure*: an assumption on the shape of the generated property sets.

8 Structural reduction: width-bounded compression

Structure enters through *scopes*: a property set over a product space $\prod_v A_v$ has support σ when it depends only on the variables in σ (`HasSupport`). Scoped constraints induce a constraint (hyper)graph, and when that graph is tree-like the feasible family compresses. The mechanization builds the full chain from scope accounting to a polynomial-size faithful representation.

8.1 Scope accounting and cut elimination

Joining constraints unions their scopes (`HasSupport.inter_union`, family form `HasSupport.iInter`); conditioning (fixing a variable) removes it from the scope and commutes with joins (`condition_inter`). The pivotal dissolution step is `cut_elimination`: if two constraints share only a cut variable v , conditioning on v leaves constraints with *disjoint* scopes — the problem falls apart into independent blocks, and joint feasibility becomes independent per-block feasibility (`condition_inter_nonempty_iff`).

The space currency is the `table`: the projection of a constraint to its scope. A supported constraint is faithfully recovered from its table (`HasSupport.eq_preimage_table`), and the table of a bag of at most $w + 1$ variables over alphabets of size at most k has at most k^{w+1} rows (`table_encard_le`, `table_encard_le_pow`).

8.2 Elimination and adaptive consistency

Bucket elimination [5, 6] is mechanized as a fold of conditioning steps. One step joins a bucket and eliminates its variable, staying within the bag bound (`elimination_step`); a whole order of width w with n steps materializes at most $n \cdot k^{w+1}$ total table rows (`elimination_order_total_bound`, `elimination_order_table_total_bound`, `bucketEliminate_total_space`). A *complete* order — one covering every scope — decides the instance: the residue is `Set.univ` or $\emptyset$ (`eliminate_eq_univ_or_empty`, bucket form `bucketEliminate_decides`), i.e. the feasibility query is answered without ever materializing the feasible family.

The fused solver statement is `projectBucketEliminate_complete_solver`: for any complete order of width w , one theorem simultaneously decides feasibility and infeasibility, extracts a concrete global solution backtrack-free when feasible, and certifies the width bound `inducedTreewidth` $\leq w$. Adaptive consistency proper — the decreasing-order run whose separators are the algorithm’s own recorded message scopes — is `decreasingRun_adaptiveSolver`.

8.3 Junction trees: a faithful polynomial representation

Elimination’s trace can be kept as a data structure: the `junctionTables` (one bag table per materialized bucket) with total size `junctionSize`. This representation is *faithful*: a scoped constraint is exactly the preimage of its bag table, and a global assignment is feasible for the whole instance iff it restricts into every bag table (`mem_joinConstraint_iff_restrict`, `junctionTables_faithful`). The size theorem `junctionTree_size_le` bounds the representation by $n \cdot k^{tw+1}$ over the induced treewidth; at fixed width the bound is linear in the number of variables (`junctionTree_size_linear`, existence form `junctionTree_size_le_card` at the minimal duplicate-free width `junctionWidth`, which dominates the induced treewidth by `inducedTreewidth_le_junctionWidth` — that the two coincide is not mechanized and remains outlook). The upshot: a feasible family of size exponential in n is represented, queried for feasibility, and sampled from, in $O(n)$ space at bounded width.

8.4 Factorization over partitions

When the constraint graph is disconnected the compression is exact and multiplicative. For blocks pulled back along coordinate projections (`pullbackFamily`), the feasible family is exactly the product of per-block families (`feasibilitySet_pullbackFamily`) and its cardinality is the product of per-block cardinalities (`encard_feasibilitySet_blocks`; two-level version `encard_feasibilitySet_blockFamilies`; domination bound `encard_feasibilitySet_blocks_le`). An

estimator pays $\sum_i \text{cost}(C_i)$ across blocks instead of $\prod_i \text{cost}(C_i)$ for the joint family: the exponential is confined inside each block.

8.5 Representation bounds: how compressible is a family?

The rectangle-cover machinery of `Ste.RepresentationBounds` quantifies compressibility in the communication-complexity sense [8]. The cover number `rectCoverNumber` — the least number of variable-separable product sets whose union is the family — is sandwiched by machine checked bounds: it is at most the family’s cardinality (`rectCoverNumber_le_encard`), at least every fooling set (`foolingNumber_le_rectCoverNumber`), and equals 1 exactly when the Čech obstruction of the singleton cover vanishes (`rectCoverNumber_eq_one_iff_cechVanishes`). On the maximally coupled family the cover number is computed exactly (`rectCoverNumber_allEqual`). These give the practitioner a diagnostic: a small fooling set in the shrunken family is a certificate that no small rectangular (per-variable factored) representation exists.

8.6 The Carlson bridge: reducing the problem class itself

Structural reduction can act one level up, on the hypothesis universe before any property set is applied. Carlson’s dissertation recasts decryption as STE [2]: keys are hypotheses, each intercepted ciphertext induces a property set (`keyPropertySet`), and the feasible key set is literally an STE feasibility set (`feasibleKeys`, `mem_feasibleKeys`), with more traffic only narrowing it (`feasibleKeys_antitone`). The residual family size after one observation is counted exactly: $(|A| - |T|)!$ keys remain consistent, where T is the observed symbol set (`card_consistent_keys`, Carlson’s Lemma 4.1), out of $|A|!$ substitution keys (`card_substitution_keys`); identification is the ideal case `Unicity`.

The reduction results then collapse cipher *classes*: every permutation-cipher key is a substitution key via the injective realization `coperm` (`permutation_reduces_to_substitution`, sharpened by `coperm_injective`), the reverse reduction is impossible for $n \geq 2$ — there are strictly more substitution keys than permutation keys, so no faithful embedding is onto (`card_permutation_lt_card_substitution`, `substitution_not_reducible_to_permutation`) — and any permutation–substitution–permutation composite is again a single substitution key (`psp_reduces_to_substitution`); cf. the same reduction in the applied literature [3, 7]. For the STE practitioner this is the template for *class-level* shrinking: prove one hypothesis class embeds in another, attack the larger class once, and inherit the feasible-family analysis for every problem in the smaller class — while the counting inequality warns which direction the reduction cannot go.

9 The lower-bound wall for structure

Structure is not optional decoration; when it is absent, blow-up is forced. The singleton-cover gluing characterization `cechVanishes_iff_rectangular` proves that margin-based local reasoning is exact — every compatible family of per-variable local sections glues — *iff* the family is a rectangle: no coupling, no obstruction. The n -fold coupling `allEqual` (all coordinates equal over an alphabet α) sits at the opposite extreme: every per-variable margin is full, so all $|\alpha|^n$ local-section families are compatible (`compatibleFamilies_allEqual_encard`), yet only the $|\alpha|$ constant vectors are actually feasible (`allEqual_encard`). The compatible-but-infeasible gap — the phantom assignments any per-variable procedure must entertain and the coupling then kills — is exactly $|\alpha|^n - |\alpha|$ (`cechObstruction_allEqual`), hence at least $2^n - 2$ once $|\alpha| \geq 2$ (`cechObstruction_allEqual_`

ge): exponential in the number of variables while the feasible family itself has constant size. For $n \geq 2$ the constraint is genuinely non-rectangular and the obstruction genuinely nonvanishing (`allEqual_not_rectangular`, `allEqual_not_cechVanishes`).

Remark 4 (Honest boundary). *What is mechanized is exactly: the iff for the singleton cover, and the exact and asymptotic obstruction counts for `allEqual`. Not mechanized (and not claimed): that `allEqual`'s induced width is minimal in the formal *Ste. Treewidth* sense, and the fully general statement that every width- w representation scheme for `allEqual` needs size $2^{\Omega(n)}$; the cohomological picture for covers with overlaps [1] remains outlook. The practical reading is nevertheless sharp: before investing in elimination orders and junction trees, measure the coupling; a complete-graph-like instance pays the exponential somewhere no matter how the pipeline is arranged.*

10 Practitioner contract

The techniques compose into a fixed order of operations for an STE attack.

1. **Generate** the property-set family from the information sources; record provenance supports.
2. **Apply** audited property sets with the exact meet (`applyConstraint`, `solve_eq_feasible`); schedule freely (idempotent, commutative); log the exact deltas (`apply_union_reduction`).
3. **Restrict / garbage-collect** across instance changes: reuse the family under corpus growth via the delta law (`mem_feasible_extension`); drop retired claims by partial restriction (`claimRestriction_exists_iff`); never invert a merge by bookkeeping (retraction asymmetry).
4. **Check feasibility**, and treat emptiness as detection of invalid information (`exists_unfair_of_feasibilitySet_eq_empty`).
5. **Collapse safely** by the must-quotient only (`mustSame_equivalence`, `CanonicalFrame`); publish must/cannot/uncertain statuses (`corefStatus`); never transitively close may-links (`maySame_not_transitive`).
6. **Certify** settled conclusions by inclusion-minimal supports (`exists_minimal_mustSupport`).
7. **Compose** shard-wise with the exact gluing obstruction (`globalExtension_nonempty_iff`); uniqueness of extension is guaranteed under compatibility (`globalExtension_subsingleton`).
8. **Compress structurally** when scopes permit: bucket elimination and adaptive consistency decide and solve within induced width (`projectBucketEliminate_complete_solver`); keep the junction-tree tables as the $O(n)$ faithful representation at fixed width (`junctionTree_size_le`); factor across disconnected blocks (`encard_feasibilitySet_blocks`); reduce hypothesis classes when an embedding is proven (`permutation_reduces_to_substitution`).
9. **Know the walls**: generically, 2^N extensional states are realizable and tight (`card_allExactStates`, `arbitrary_subset_as_single_property`); structurally, maximal coupling forces an obstruction of at least $2^n - 2$ (`cechObstruction_allEqual_ge`).

Finally, the preservation ledger — which queries each step provably preserves:

Step	Preserves	Authority
Exact meet	all queries (it <i>defines</i> them)	<code>solve_eq_feasible</code>
Restriction ($D \subseteq E$)	must/cannot upward; may/uncertain downward	<code>mustSame_mono</code> , <code>maySame_anti</code>
Must-quotient	must; statuses of remaining pairs	<code>coreference_exhaustive</code>
Minimal support	the certified must/cannot fact	<code>exists_minimal_mustSupport</code>
Gluing	feasibility on the union, exactly	<code>mem_feasible_union_iff</code>
Elimination (complete order)	feasibility + one witness	<code>projectBucketEliminate_com</code>
Junction tables	full membership (hence all queries)	<code>mem_joinConstraint_iff_res</code>
Factorization	full family, as a product	<code>feasibilitySet_pullbackFam</code>

Restriction deserves the final caution, because its ledger row is asymmetric by theorem, not by implementation: settled must/cannot facts persist as the corpus grows (`mustSame_mono`, `cannotSame_mono`) but may be lost under deletion, while may/uncertain facts transfer in the opposite direction only (`maySame_antitone`, `maySeparate_antitone`, `uncertain_antitone`). A pipeline that caches query answers across corpus edits must cache them with the direction of the edit — which is, in the end, the whole discipline this note proposes: shrink as hard as the theorems allow, and no harder.

References

- [1] Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13:113036, 2011. doi: 10.1088/1367-2630/13/11/113036. arXiv:1102.0264.
- [2] Albert H. Carlson. *Set Theoretic Estimation Applied to the Information Content of Ciphers and Decryption*. Ph.d. dissertation, University of Idaho, Moscow, Idaho, USA, May 2012. Major professors R. E. Hiromoto and R. B. Wells. Retrieved via Internet Archive snapshot (2024-07-13) of the author’s ResearchGate full text, publication 349064680.
- [3] Albert H. Carlson, Sarangi R. Mikkilineni, Michael W. Totaro, Richard B. Wells, and Robert E. Hiromoto. Equivalence of product ciphers to substitution ciphers and their security implications. In *IEEE Int. Symp. on Networks, Computers and Communications (ISNCC)*, 2022. doi: 10.1109/ISNCC55209.2022.9851719.
- [4] Patrick L. Combettes. The foundations of set theoretic estimation. *Proceedings of the IEEE*, 81(2):182–208, 1993. doi: 10.1109/5.214546. Retrieved from the author’s page, <https://pcombet.math.ncsu.edu/proc.pdf>.
- [5] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1–2):41–85, 1999. doi: 10.1016/S0004-3702(99)00059-4.
- [6] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, San Francisco, 2003.
- [7] Bhaskar Ghosh, Indira Kalyan Dutta, Shivam Khare, Albert Carlson, and Michael Totaro. Isomorphic cipher reduction. In *IEEE Int. Conf. on Information Technology, Electronics and Mobile Communication (IEMCON)*, pages 947–953, 2021. doi: 10.1109/IEMCON53756.2021.9623135.
- [8] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, Cambridge, 1997. doi: 10.1017/CBO9780511574948. ISBN 978-0-521-56067-2.